

Product / Growth Analytics

Funnel · Cohort · Churn Analysis

Formulas | Worked Examples | SQL Queries | Python Code

Compiled reference notes — complete guide with SQL & Python implementations.

1. Funnel Analysis

A funnel tracks how users move through a sequence of steps (e.g. Visit → Signup → Add to Cart → Payment → Purchase) and where they drop off along the way.

1.1 Core Formulas

1. Conversion Rate (step-to-step)

$$\text{CR (\%)} = (\text{Current Step Users} / \text{Previous Step Users}) \times 100$$

2. Overall Funnel Conversion

$$\text{Overall CR (\%)} = (\text{Final Step Users} / \text{First Step Users}) \times 100$$

3. Drop-off Count

$$\text{Drop-off} = \text{Previous Step Users} - \text{Current Step Users}$$

4. Drop-off Rate

$$\text{Drop-off (\%)} = (\text{Previous Step} - \text{Current Step}) / \text{Previous Step} \times 100$$

5. Retention Rate (per step)

$$\text{Retention (\%)} = (\text{Current Step Users} / \text{Previous Step Users}) \times 100$$

6. Revenue per Visitor

$$\text{Revenue/Visitor} = \text{Total Revenue} / \text{Total Visitors}$$

7. Exit Rate

$$\text{Exit (\%)} = (\text{Users Exiting on Page} / \text{Users Visiting Page}) \times 100$$

8. Purchase Conversion

$$\text{Purchase (\%)} = (\text{Purchases} / \text{Visitors}) \times 100$$

9. Payment Success Rate

$$\text{Payment Success (\%)} = (\text{Successful Payments} / \text{Payment Attempts}) \times 100$$

10. Funnel Velocity (Average Time to Convert)

$$\text{Avg Time} = \text{Total Time Taken} / \text{Completed Users}$$

11. Average Order Value (AOV)

$$\text{AOV} = \text{Total Revenue} / \text{Number of Orders}$$

12. Average Revenue Per User (ARPU)

$$\text{ARPU} = \text{Total Revenue} / \text{Total Users}$$

13. Cost Per Acquisition (CPA)

$$\text{CPA} = \text{Total Marketing Cost} / \text{Customer Acquisitions}$$

14. Customer Acquisition Cost (CAC)

$$\text{CAC} = \text{Total Acquisition Cost} / \text{New Customers}$$

15. Customer Lifetime Value (CLV)

$$\text{CLV} = \text{Average Revenue Per Customer} \times \text{Average Customer Lifetime}$$

1.2 Worked Example

Suppose an e-commerce app tracks 10,000 visitors through 5 funnel steps in a month:

Step	Users	Conversion from Previous	Drop-off
1. Website Visit	10,000	—	—
2. Product View	6,500	65.0%	3,500
3. Add to Cart	2,600	40.0%	3,900
4. Checkout Started	1,300	50.0%	1,300
5. Payment Success	910	70.0%	390

Overall Funnel Conversion = $910 / 10,000 \times 100 = 9.1\%$

Biggest drop-off: Product View → Add to Cart (3,900 users, 60% drop), indicating a cart-addition friction problem.

1.3 SQL Query — Funnel Step Counts & Conversion

SQL — Step-wise funnel using conditional aggregation

```
-- events table: user_id, event_name, event_time
WITH funnel AS (
    SELECT
        COUNT(DISTINCT CASE WHEN event_name = 'website_visit' THEN user_id END) AS step1_visit,
        COUNT(DISTINCT CASE WHEN event_name = 'product_view' THEN user_id END) AS step2_view,
        COUNT(DISTINCT CASE WHEN event_name = 'add_to_cart' THEN user_id END) AS step3_cart,
        COUNT(DISTINCT CASE WHEN event_name = 'checkout_start' THEN user_id END) AS step4_checkout,
        COUNT(DISTINCT CASE WHEN event_name = 'payment_success' THEN user_id END) AS step5_paid
    FROM events
    WHERE event_time BETWEEN '2026-07-01' AND '2026-07-31'
)
SELECT
    step1_visit,
    step2_view,
    ROUND(step2_view * 100.0 / NULLIF(step1_visit, 0), 2) AS cr_visit_to_view,
    step3_cart,
    ROUND(step3_cart * 100.0 / NULLIF(step2_view, 0), 2) AS cr_view_to_cart,
    step4_checkout,
    ROUND(step4_checkout * 100.0 / NULLIF(step3_cart, 0), 2) AS cr_cart_to_checkout,
    step5_paid,
    ROUND(step5_paid * 100.0 / NULLIF(step4_checkout, 0), 2) AS cr_checkout_to_paid,
    ROUND(step5_paid * 100.0 / NULLIF(step1_visit, 0), 2) AS overall_conversion
FROM funnel;
```

SQL — Sequential (ordered) funnel using window functions

```
-- Ensures each step only counts users who did prior steps in order
WITH ranked AS (
    SELECT user_id, event_name, event_time,
        ROW_NUMBER() OVER (PARTITION BY user_id, event_name ORDER BY event_time) AS rn
    FROM events
),
first_events AS (
    SELECT user_id, event_name, event_time FROM ranked WHERE rn = 1
),
step_flags AS (
    SELECT
        user_id,
        MAX(CASE WHEN event_name = 'website_visit' THEN 1 ELSE 0 END) AS s1,
        MAX(CASE WHEN event_name = 'product_view' THEN 1 ELSE 0 END) AS s2,
        MAX(CASE WHEN event_name = 'add_to_cart' THEN 1 ELSE 0 END) AS s3,
        MAX(CASE WHEN event_name = 'checkout_start' THEN 1 ELSE 0 END) AS s4,
        MAX(CASE WHEN event_name = 'payment_success' THEN 1 ELSE 0 END) AS s5
    FROM first_events
```

```

        GROUP BY user_id
    )
SELECT
    SUM(s1) AS visit,
    SUM(CASE WHEN s1=1 AND s2=1 THEN 1 ELSE 0 END) AS viewed,
    SUM(CASE WHEN s2=1 AND s3=1 THEN 1 ELSE 0 END) AS carted,
    SUM(CASE WHEN s3=1 AND s4=1 THEN 1 ELSE 0 END) AS checkout,
    SUM(CASE WHEN s4=1 AND s5=1 THEN 1 ELSE 0 END) AS paid
FROM step_flags;

```

1.4 Python Code — Funnel Analysis with Pandas

Python — build funnel table, conversion & drop-off, and a Plotly funnel chart

```

import pandas as pd
import plotly.graph_objects as go

# events: user_id, event_name, event_time
events = pd.read_csv("events.csv", parse_dates=["event_time"])

steps = ["website_visit", "product_view", "add_to_cart", "checkout_start", "payment_success"]

funnel_counts = []
for step in steps:
    n_users = events.loc[events["event_name"] == step, "user_id"].nunique()
    funnel_counts.append(n_users)

funnel_df = pd.DataFrame({"step": steps, "users": funnel_counts})
funnel_df["conversion_%"] = (funnel_df["users"] / funnel_df["users"].shift(1) * 100).round(2)
funnel_df.loc[0, "conversion_%"] = 100.0
funnel_df["overall_%"] = (funnel_df["users"] / funnel_df["users"].iloc[0] * 100).round(2)
funnel_df["drop_off"] = funnel_df["users"].shift(1) - funnel_df["users"]
funnel_df["drop_off_%"] = (funnel_df["drop_off"] / funnel_df["users"].shift(1) * 100).round(2)

print(funnel_df)

# Visualize
fig = go.Figure(go.Funnel(y=funnel_df["step"], x=funnel_df["users"]))
fig.update_layout(title="Purchase Funnel")
fig.write_html("funnel_chart.html")

```

2. Cohort Analysis

Cohort analysis groups users by a shared starting point (e.g. signup month) and tracks their behaviour — usually retention — over time, revealing whether a product is actually 'sticky'.

2.1 Core Formulas

1. Cohort Retention Rate

Cohort Retention (%) = (Users Active in Period N / Users in Original Cohort) × 100

2. Retention Rate (general)

Retention (%) = (Remaining Users / Original Users) × 100

3. Churned Users in a Cohort

Churned = Original Cohort Users – Active Users in Period N

4. Cohort-based CLV

Cohort CLV = ARPU × Average Customer Lifetime (for that cohort)

2.2 Worked Example

Signup cohort of **January 2026**, tracked monthly:

Cohort	Month 0	Month 1	Month 2	Month 3
Jan-2026 (1,000 users)	100%	45%	30%	22%
Feb-2026 (1,200 users)	100%	50%	35%	—
Mar-2026 (900 users)	100%	48%	—	—

Reading it: Month-1 retention improved from 45% (Jan cohort) to 50% (Feb cohort) — a product/onboarding change made in February likely helped. Month-2 retention is also higher for Feb (35%) vs Jan (30%), confirming the improvement held.

2.3 SQL Query — Monthly Cohort Retention Table

SQL — classic cohort retention grid

```
-- users: user_id, signup_date
-- activity: user_id, activity_date

WITH cohort AS (
  SELECT user_id,
         DATE_TRUNC('month', signup_date) AS cohort_month
  FROM users
),
activity_month AS (
  SELECT a.user_id,
         DATE_TRUNC('month', a.activity_date) AS activity_month
  FROM activity a
  GROUP BY a.user_id, DATE_TRUNC('month', a.activity_date)
),
cohort_activity AS (
  SELECT
    c.cohort_month,
    am.activity_month,
    DATE_PART('month', AGE(am.activity_month, c.cohort_month)) AS period_number,
    COUNT(DISTINCT c.user_id) AS active_users
  FROM cohort c
```

```

        JOIN activity_month am ON am.user_id = c.user_id
        GROUP BY c.cohort_month, am.activity_month
    ),
    cohort_size AS (
        SELECT cohort_month, COUNT(DISTINCT user_id) AS num_users
        FROM cohort
        GROUP BY cohort_month
    )
SELECT
    ca.cohort_month,
    ca.period_number,
    ca.active_users,
    cs.num_users AS cohort_size,
    ROUND(ca.active_users * 100.0 / cs.num_users, 2) AS retention_pct
FROM cohort_activity ca
JOIN cohort_size cs ON cs.cohort_month = ca.cohort_month
ORDER BY ca.cohort_month, ca.period_number;

```

2.4 Python Code — Cohort Retention Heatmap

Python — build cohort retention matrix with Pandas + Seaborn heatmap

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# users: user_id, signup_date
# activity: user_id, activity_date
users = pd.read_csv("users.csv", parse_dates=["signup_date"])
activity = pd.read_csv("activity.csv", parse_dates=["activity_date"])

users["cohort_month"] = users["signup_date"].values.astype("datetime64[M]")
activity = activity.merge(users[["user_id", "cohort_month"]], on="user_id")
activity["activity_month"] = activity["activity_date"].values.astype("datetime64[M]")

activity["period_number"] = (
    (activity["activity_month"].dt.year - activity["cohort_month"].dt.year) * 12
    + (activity["activity_month"].dt.month - activity["cohort_month"].dt.month)
)

cohort_data = (activity.groupby(["cohort_month", "period_number"])["user_id"]
               .nunique().reset_index())

cohort_pivot = cohort_data.pivot(index="cohort_month", columns="period_number", values="user_id")
cohort_size = cohort_pivot.iloc[:, 0]
retention = cohort_pivot.divide(cohort_size, axis=0).round(3) * 100

print(retention)

plt.figure(figsize=(10, 6))
sns.heatmap(retention, annot=True, fmt=".1f", cmap="Blues")
plt.title("Monthly Cohort Retention (%)")
plt.ylabel("Signup Cohort")
plt.xlabel("Months Since Signup")
plt.tight_layout()
plt.savefig("cohort_retention_heatmap.png")

```

3. Churn Analysis

Churn analysis measures how many customers (or how much revenue) you are losing over a period, and is used to predict and prevent future churn.

3.1 Core Formulas

1. Churn Rate

$$\text{Churn Rate (\%)} = (\text{Customers Lost} / \text{Customers at Start}) \times 100$$

2. Retention Rate

$$\text{Retention (\%)} = (\text{Ending Customers} - \text{New Customers}) / \text{Beginning Customers} \times 100$$

3. Average Revenue Per User (ARPU)

$$\text{ARPU} = \text{Total Revenue} / \text{Total Users}$$

4. Customer Acquisition Cost (CAC)

$$\text{CAC} = (\text{Marketing Cost} + \text{Sales Cost}) / \text{New Customers}$$

5. Customer Lifetime Value (CLV)

$$\text{CLV} = \text{ARPU} \times \text{Average Customer Lifetime}$$

6. CLV : CAC Ratio

Healthy benchmark $\approx 3 : 1$ | $1 : 1$ or lower is unhealthy (losing money per customer)

7. Revenue Churn Rate

$$\text{Revenue Churn (\%)} = (\text{Lost Revenue} / \text{Starting Revenue}) \times 100$$

8. Net Revenue Retention (NRR)

$$\text{NRR (\%)} = (\text{Starting Revenue} + \text{Expansion} - \text{Contraction} - \text{Churned Revenue}) / \text{Starting Revenue} \times 100$$

9. Cohort Retention

$$\text{Retention (\%)} = (\text{Remaining Customers} / \text{Original Cohort}) \times 100$$

Churn Prediction (ML): a classification model (e.g. Logistic Regression, Random Forest, XGBoost) trained on customer features (tenure, usage, support tickets, payment history, plan type) to predict probability of churn — see the Python example below.

3.2 Worked Example

Metric	Value
Customers at start of month	5,000
Customers lost during month	250
New customers added	400
Customers at end of month	5,150
Churn Rate	$250 / 5,000 \times 100 = 5.0\%$
Retention Rate	$(5,150 - 400) / 5,000 \times 100 = 95.0\%$

Starting MRR	■10,00,000
Churned MRR	■45,000
Revenue Churn Rate	$45,000 / 10,00,000 \times 100 = 4.5\%$

A 5% monthly customer churn compounds to roughly **≈46% annual churn** ($1 - (1 - 0.05)^{12}$), which is why even small monthly churn numbers deserve attention.

3.3 SQL Query — Monthly Churn & Retention

SQL — churned, retained and new customers per month

```
-- subscriptions: user_id, status, start_date, end_date

WITH monthly_status AS (
    SELECT
        DATE_TRUNC('month', d.month) AS month,
        s.user_id,
        CASE WHEN s.start_date <= d.month AND (s.end_date IS NULL OR s.end_date > d.month)
            THEN 1 ELSE 0 END AS is_active
    FROM subscriptions s
    CROSS JOIN (SELECT generate_series('2026-01-01'::date, '2026-12-01'::date, '1 month') AS month) d
),
agg AS (
    SELECT month, user_id, MAX(is_active) AS active
    FROM monthly_status
    GROUP BY month, user_id
),
month_pairs AS (
    SELECT
        a.month AS curr_month,
        a.user_id,
        a.active AS active_this_month,
        LAG(a.active) OVER (PARTITION BY a.user_id ORDER BY a.month) AS active_last_month
    FROM agg a
)
SELECT
    curr_month,
    COUNT(*) FILTER (WHERE active_last_month = 1) AS customers_at_start,
    COUNT(*) FILTER (WHERE active_last_month = 1 AND active_this_month = 0) AS churned,
    COUNT(*) FILTER (WHERE active_last_month = 0 AND active_this_month = 1) AS new_customers,
    ROUND(
        COUNT(*) FILTER (WHERE active_last_month = 1 AND active_this_month = 0) * 100.0
        / NULLIF(COUNT(*) FILTER (WHERE active_last_month = 1), 0), 2
    ) AS churn_rate_pct
FROM month_pairs
GROUP BY curr_month
ORDER BY curr_month;
```

3.4 Python Code — Churn Rate Calculation

Python — monthly churn & retention rate from subscription data

```
import pandas as pd

subs = pd.read_csv("subscriptions.csv", parse_dates=["start_date", "end_date"])
months = pd.date_range("2026-01-01", "2026-12-01", freq="MS")

rows = []
for m in months:
    active_start = subs[(subs["start_date"] <= m) &
                        ((subs["end_date"].isna()) | (subs["end_date"] > m))]["user_id"].nunique()

    next_m = m + pd.offsets.MonthBegin(1)
    churned = subs[(subs["start_date"] <= m) &
                   (subs["end_date"] > m) & (subs["end_date"] <= next_m)]["user_id"].nunique()
    new_customers = subs[(subs["start_date"] > m) & (subs["start_date"] <= next_m)]["user_id"].nunique()
```



```

churn_rate = round(churned / active_start * 100, 2) if active_start else 0
retention_rate = round((active_start - churned) / active_start * 100, 2) if active_start else 0

rows.append({"month": m, "customers_at_start": active_start, "churned": churned,
            "new_customers": new_customers, "churn_rate_%": churn_rate,
            "retention_rate_%": retention_rate})

churn_df = pd.DataFrame(rows)
print(churn_df)

```

3.5 Python Code — Churn Prediction (Machine Learning)

Python — Logistic Regression / Random Forest churn prediction model

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score

# customers.csv: tenure_months, monthly_spend, support_tickets,
#                 num_logins_last_30d, plan_type (encoded), churned (0/1)
df = pd.read_csv("customers.csv")

X = df.drop(columns=["churned", "user_id"])
y = df["churned"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# Baseline: Logistic Regression
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_s, y_train)
log_preds = log_model.predict(X_test_s)
print("Logistic Regression:")
print(classification_report(y_test, log_preds))
print("AUC:", roc_auc_score(y_test, log_model.predict_proba(X_test_s)[: , 1]))

# Stronger model: Random Forest
rf_model = RandomForestClassifier(n_estimators=300, max_depth=8, random_state=42)
rf_model.fit(X_train, y_train)
rf_preds = rf_model.predict(X_test)
print("\nRandom Forest:")
print(classification_report(y_test, rf_preds))
print("AUC:", roc_auc_score(y_test, rf_model.predict_proba(X_test)[: , 1]))

# Feature importance -> what drives churn
importance = pd.Series(rf_model.feature_importances_, index=X.columns).sort_values(ascending=False)
print("\nTop churn drivers:\n", importance.head(10))

```

3.6 Quick Reference — All Formulas at a Glance

Area	Metric	Formula
Funnel	Conversion Rate	Current Step / Previous Step × 100
Funnel	Overall Conversion	Final Step / First Step × 100
Funnel	Drop-off Rate	(Prev – Curr) / Prev × 100
Funnel	Exit Rate	Exit Page Users / Page Visitors × 100

Funnel	AOV	Revenue / Orders
Funnel	ARPU	Revenue / Total Users
Cohort	Cohort Retention	Active in Period N / Original Cohort × 100
Churn	Churn Rate	Customers Lost / Customers at Start × 100
Churn	Retention Rate	(Ending – New) / Beginning × 100
Churn	CAC	(Marketing + Sales) / New Customers
Churn	CLV	ARPU × Avg Customer Lifetime
Churn	CLV : CAC	3:1 healthy, 1:1 unhealthy
Churn	Revenue Churn	Lost Revenue / Starting Revenue × 100
Churn	NRR	(Start + Expansion – Contraction – Churn) / Start × 100